

TileLang & PyPTO 助力 DeepSeek-V4 快速适配与调优

作者：韩旭 王子楠

时间：2026年4月29日

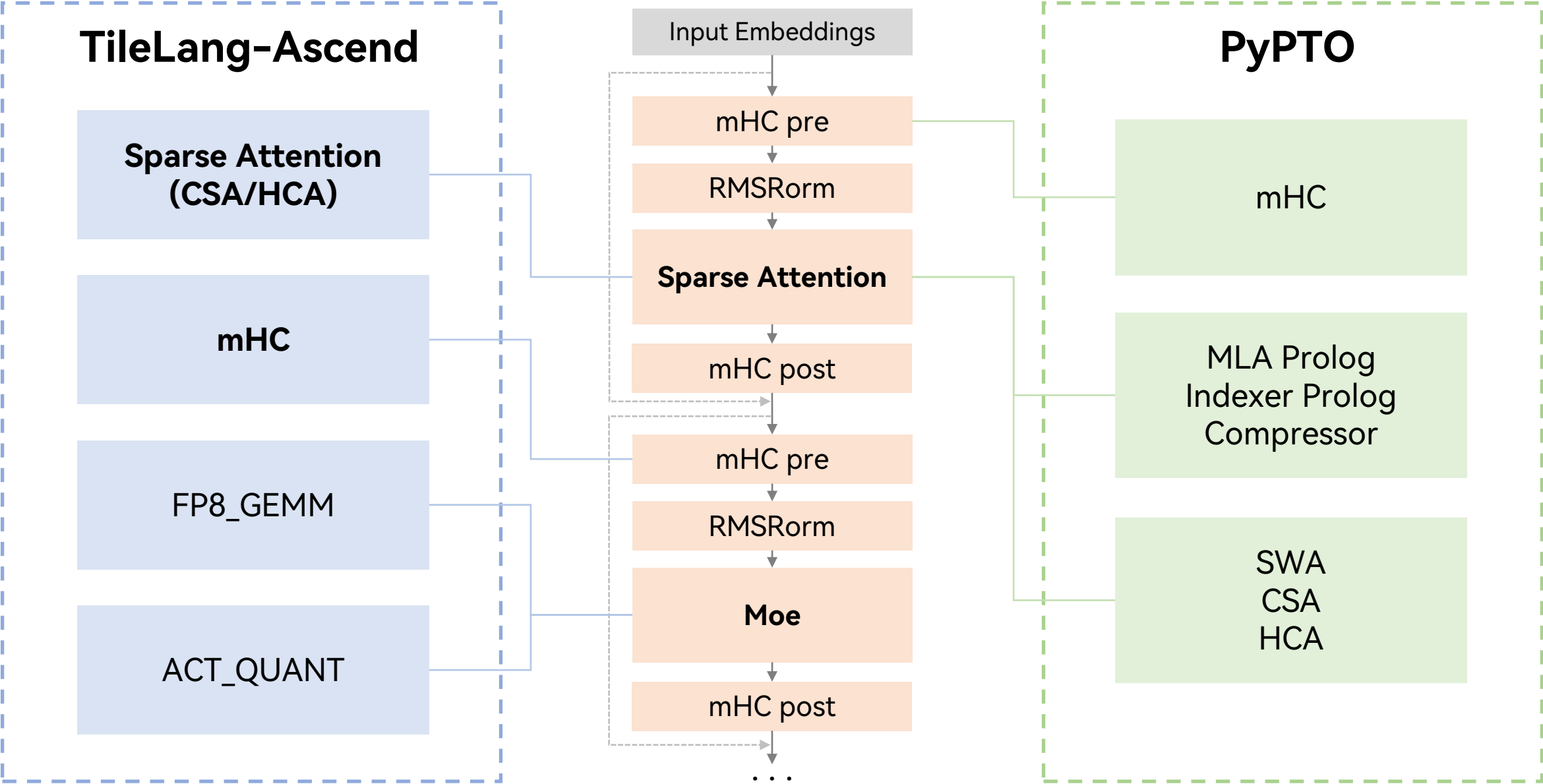
目录

Part1 整体介绍

Part2 主要算子实现

Part3 算子性能与调优

TileLang 与 PyPTO - DeepSeek-V4 模型算子实现



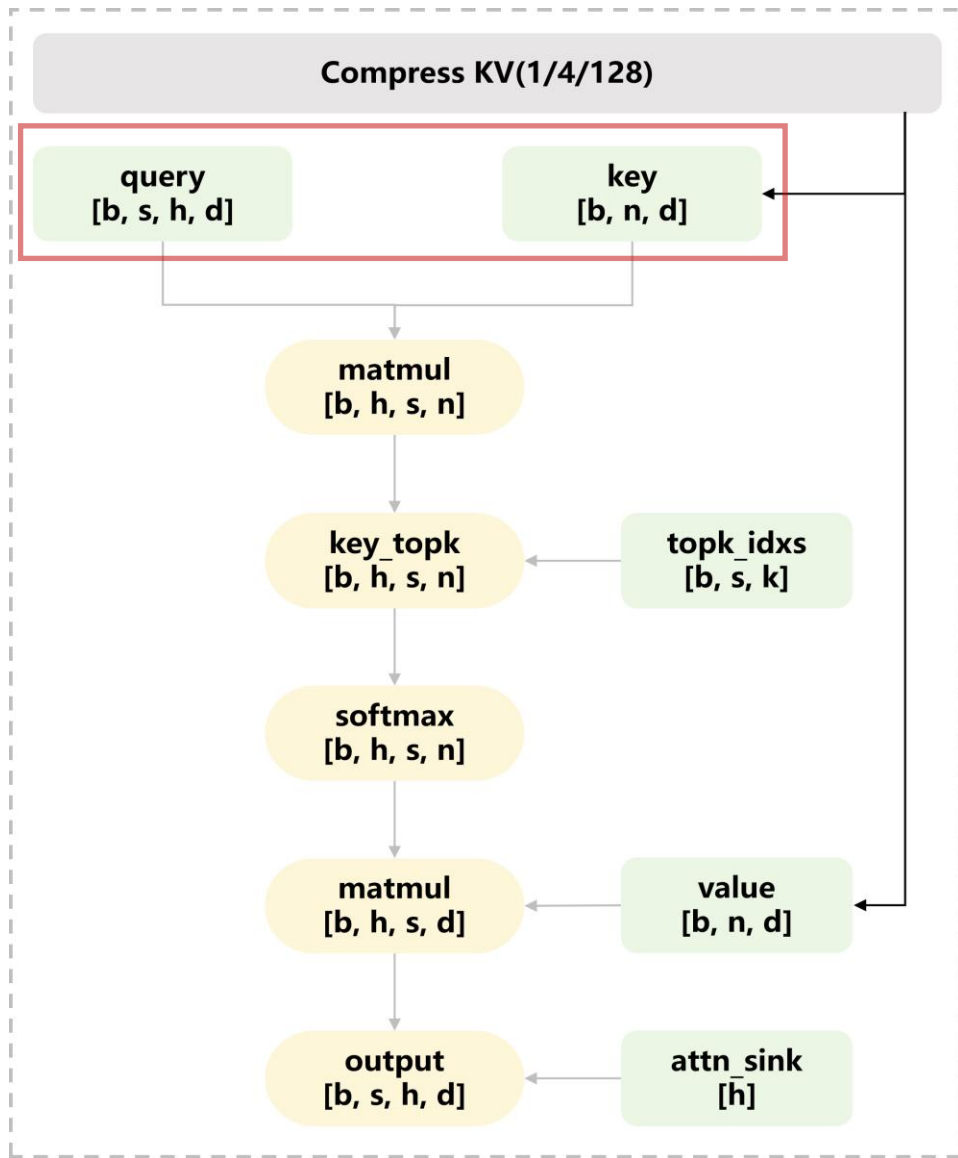
目录

Part1 整体介绍

Part2 主要算子实现

Part3 算子性能与调优

TileLang 主要算子实现: Sparse Attention



Sparse Attention 算子计算流程图

算子初始化

kv-cache收集

注意力计算

算子结果处理

算子数据切分

灵活分配核间/核内数据。

```
with T.Kernel(block_num, is_npu=True) as (cid, vid):  
    block_x = cid % m  
    ...
```

Buffer申请与初始化

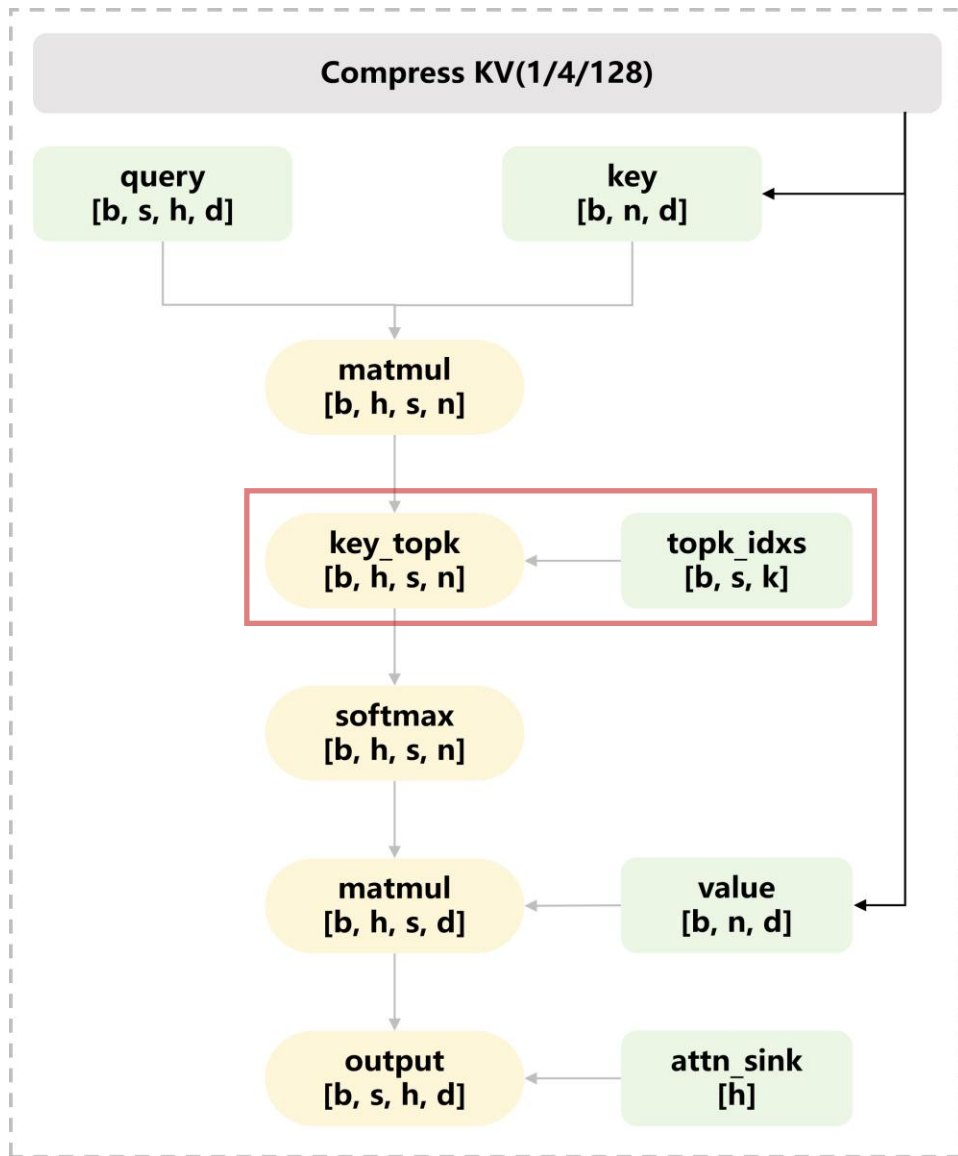
维度+数据类型格式申请 Buffer, 自动规划地址与 Buffer 复用。

```
q_l1/kv_l1... = T.alloc_L1([h, d], dtype)  
acc_s_l0c = T.alloc_L0C([h, block], accum_dtype)  
acc_s_ub = T.alloc_ub([v_block, block], accum_dtype)
```

一行指令按需初始化填充。

```
T.tile.fill(acc_o_ub, 0.0)
```

TileLang 主要算子实现: Sparse Attention



Sparse Attention 算子计算流程图

算子初始化

kv-cache收集

注意力计算

算子结果处理

Top-K索引收集

逐块收集稀疏kv-index, 并处理超出边界的索引。

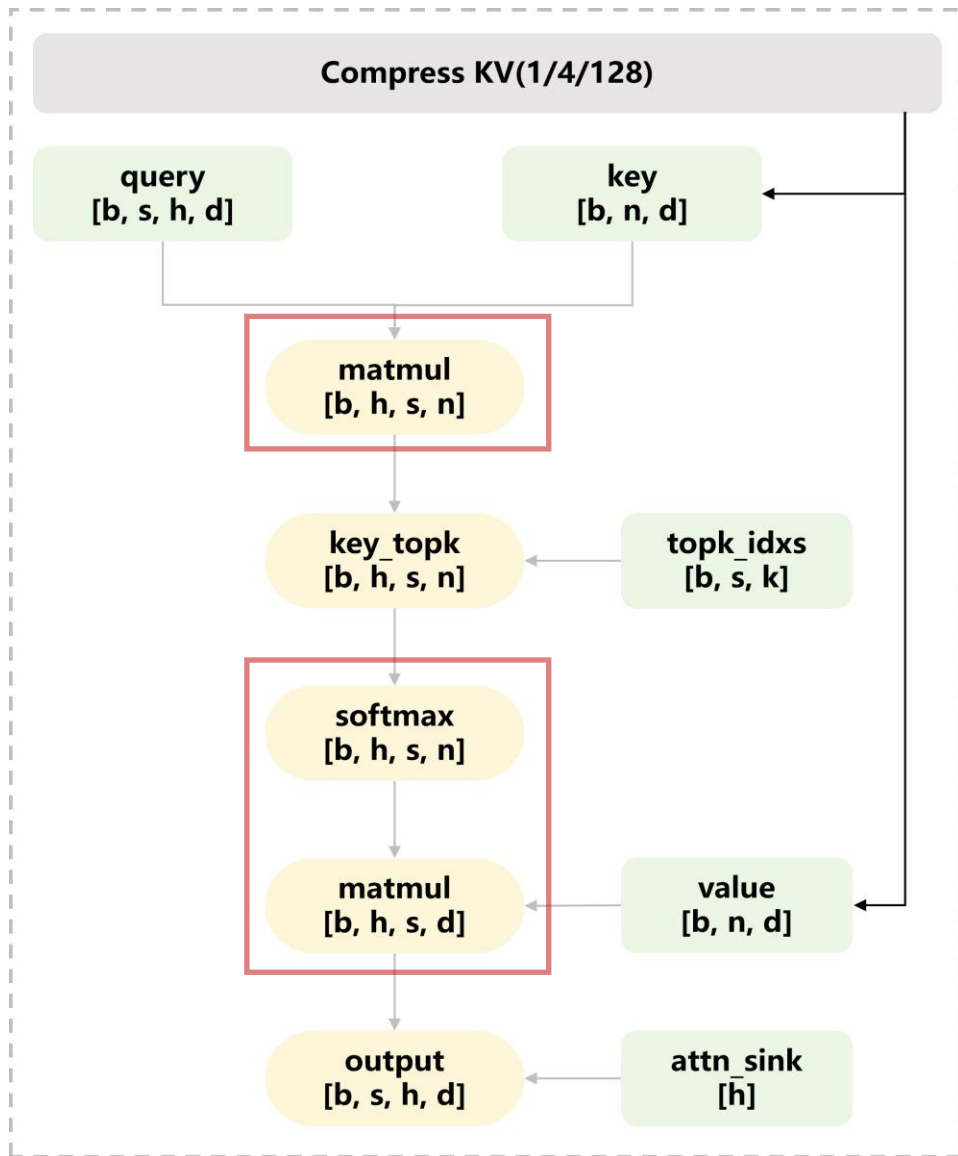
```
T.copy(topk_idx[by, bx, t*block:t*block+block], idxs_ub)
for i in T.serial(block):
    if t*block+i >= topk:
        idxs_ub[i] = -1
```

kv向量存储

根据索引, 逐块取出kv值到缓存。

```
for b_i in T.serial(block):
    idx_num = idxs_ub[b_i]
    if idx_num != -1:
        T.copy(kv[by, idx_num, :], kv_ub[b_i, :])
```

TileLang 主要算子实现: Sparse Attention



Sparse Attention 算子计算流程图

算子初始化

kv-cache收集

注意力计算

算子结果处理

$$\text{Attention}(Q, K, V) = \text{Softmax} \left(\frac{Q \cdot K^T}{\sqrt{d_k}} \right) \cdot V$$

注意力矩阵乘法

拷贝Query、Key向量，执行矩阵乘。

```
T.copy(q[by, bx, :, :], q_l1)
T.copy(workspace_1[cid, 0:block, 0:d], kv_l1)
T.gemm_v0(q_l1, kv_l1, acc_s_l0c, transpose_B=True, init=True)
T.copy(acc_s_l0c, workspace_2[cid, 0:h, 0:block])
```

Softmax计算

缩放注意力分数。

```
T.tile.mul(acc_s_ub, acc_s_ub, scale)
```

Tile级编程Softmax归一化计算。

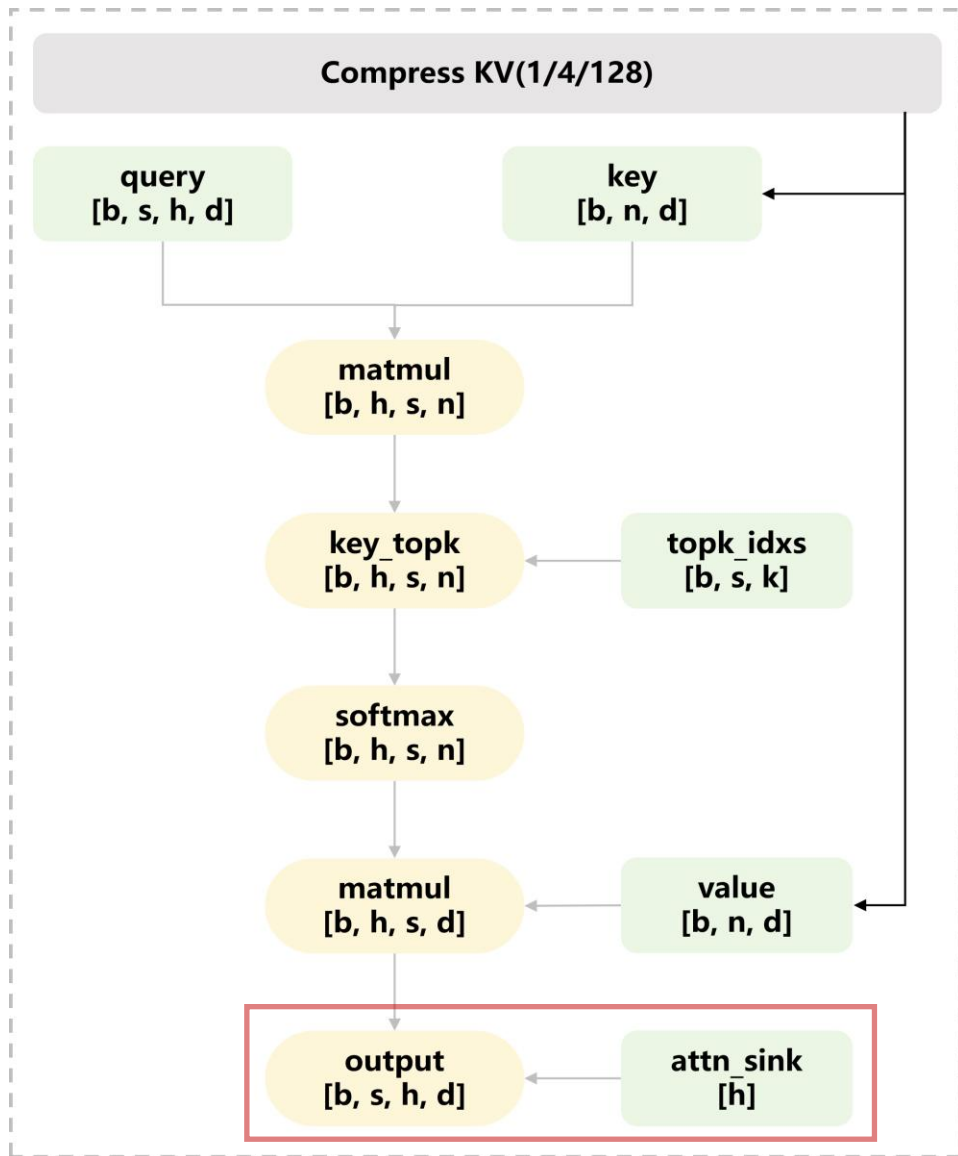
```
T.tile.add/mul/div/exp ...
```

```
T.reduce_max/reduce_sum ...
```

Softmax结果聚合Value向量。

```
T.gemm_v0(acc_s_l1, kv_l1, acc_o_l0c, init=True)
```

TileLang 主要算子实现: Sparse Attention



Sparse Attention 算子计算流程图

算子初始化

kv-cache收集

注意力计算

算子结果处理

Attention Sink 稳定性增强

对结果进行attn_sink处理, 保证注意力沉降。

```
T.tile.sub(attn_sink_ub, attn_sink_ub, scores_max[:, 0])
T.tile.exp(attn_sink_ub, attn_sink_ub)
T.tile.add(sum_exp[:, 0], sum_exp[:, 0], attn_sink_ub)
```

算子输出处理

逐块写回GM注意力值计算结果。

```
T.copy(acc_o_ub, output[by, bx,
    vid*v_block:vid*v_block+v_block, :])
```

算子调用得到output。

```
output = func(q, kv, attn_sink, topk_idx)
```


PyPTO 算子实现：针对 DeepSeek V4 的 Mla Prolog

Mla Prolog 代码片段展示

以 $q = \text{RmsNorm}(c^Q @ wq_b)$ 计算为例:

```
def rms_norm(input_tensor: pypto.Tensor, epsilon: float) -> pypto.Tensor:
    input_fp32 = pypto.cast(input_tensor, pypto.DT_FP32)
    dim = len(input_tensor.shape)
    y = pypto.mul(input_fp32, input_fp32)
    y = pypto.mul(y, 1.0 / input_tensor.shape[dim - 1])
    y = pypto.sum(y, -1, keepdim=True)
    y = pypto.add(y, epsilon)
    y = pypto.sqrt(y)
    ones_vector = pypto.full(y.shape, 1.0, pypto.DT_FP32)
    y = pypto.div(ones_vector, y)
    y = pypto.mul(input_fp32, y)
    return y

pypto.set_cube_tile_shapes([tile_bs, tile_bs], [128, 1024, 512], [256, 256], True)
qb = pypto.matmul(qr_quant, wq_b, pypto.DataType.DT_INT32)
pypto.set_vec_tile_shapes(4, 4096)
qb_dequant = dequant(qb, qr_scale, wq_b_scale)
q_3d = pypto.reshape(qb_dequant, [t_tile, head_num, head_dim])
pypto.set_vec_tile_shapes(4, 8, 512)
qr2_3d = rms_norm(q_3d, attrs.eps)
qr2_3d_cast = pypto.cast(qr2_3d, pypto.DataType.DT_BF16)
```

新版本计算流

$$\begin{aligned} c^Q &= \text{RmsNorm}(x @ wq_a) \\ q &= \text{RmsNorm}(c^Q @ wq_b) \\ q[\dots, -\text{rope_dim}:] &= \text{ROPE}(q[\dots, -\text{rope_dim}:]) \\ k &= \text{RmsNorm}(x @ wkv) \\ k[\dots, -\text{rope_dim}:] &= \text{ROPE}(k[\dots, -\text{rope_dim}:]) \end{aligned}$$

新老版本差异

DS V4 实现	DS V3.2 实现	核心差异点
Query 方向进行两次 Linear 计算	Query 方向进行三次 Linear 计算	Linear 计算次数不同
RoPE 对 Query、Key 最后 64 维度进行 inplace RoPE 操作	Query、key 均有独立的 RoPE 操作分支	RoPE 计算流差异
Query 方向需要两次 RmsNorm 操作，其中第二次 RmsNorm 权重恒为1	Query 方向上仅有一次 RmsNorm 操作	Query 归一化操作次数不同

PyPTO 算子实现: hc_pre

hc_pre 代码片段展示

以 Prefill 场景的 comb 计算为例:

```
def hc_split_sinhorn_trans(comb_flag: pypto.Tensor, hc_split_sinhorn_iters, hc_eps):  
    sinkhorn_iters = hc_split_sinhorn_iters  
    _, _, tile_t = comb_flag.shape # (4, 4, tile_t)  
    pypto.set_vec_tile_shapes(8, 16, 128)  
    row_max = pypto.amax(comb_flag, -2, True) # (4, 1, tile_t)  
    comb_flag = pypto.exp(comb_flag - row_max) # (4, 4, tile_t)  
    row_sum = pypto.sum(comb_flag, -2, True) # (4, 1, tile_t)  
    comb_flag = comb_flag / row_sum + hc_eps # (4, 4, tile_t)  
    col_sum = pypto.sum(comb_flag, -3, True) # (1, 4, tile_t)  
    comb_flag = comb_flag / (col_sum + hc_eps) # (4, 4, tile_t)
```

```
for _ in range(sinhorn_iters - 1):  
    row_sum = comb_flag.sum(-2, keepdim=True) # (4, 1, tile_t)  
    comb_flag = comb_flag / (row_sum + hc_eps) # (4, 4, tile_t)  
    col_sum = comb_flag.sum(-3, keepdim=True) # (1, 4, tile_t)  
    comb_flag = comb_flag / (col_sum + hc_eps) # (4, 4, tile_t)  
return comb_flag
```

转置后的 Sinkhorn-Knopp 算法, 归约轴变为 -2, -3 轴

```
rms_res = rms_norm_denom(x_fp32) # (t, hc*d) -> (t, 1)
```

```
mm_res = pypto.matmul(hc_fn, x_fp32, pypto.DT_FP32, b_trans=True)
```

矩阵乘转置将 t 轴置于尾轴

```
rms_res = mm_res / rms_res  
comb_flag = (rms_res[2*hc:, :] * (hc_scale_hc[:, 2:3]) + hc_base[2*hc:, :]) # (16, tile_t)  
comb_flag = comb_flag.reshape([hc, hc, tile_t]) # (4, 4, tile_t)  
comb_ = hc_split_sinhorn_trans(comb_flag, hc_split_sinhorn_iters, hc_eps) # (4, 4, tile_t)
```

```
comb_ = comb_.transpose(1, 2)  
pypto.set_vec_tile_shapes(4, 128, 4)  
comb_ = comb_.transpose(0, 1)
```

最后将结果转置, pass 将该运算与前序计算融合

hc_pre 计算流

$$rsqrt = \frac{1}{\sqrt{\frac{1}{n} \sum_{i=1}^n x_i^2 + \epsilon}}$$

$$mixes = (x @ hc_fn) \odot rsqrt$$

$$pre, post = sigmoid(mixes, hc_scale, hc_base, hc_mult, hc_sinhorn_iters)$$

$$comb = sinkhorn_knopp(mixes, hc_scale, hc_base, hc_mult, hc_sinhorn_iters)$$

$$y = rowsum(pre \odot x)$$

Prefill 等价计算实现高性能

- Prefill 场景下, x 的动态轴 t 通常比较大; 在 Sinkhorn-Knopp 算法中, 归约轴很小为 4, 归约时为 -1 轴, 列归约时为 -2 轴。
- 我们采用数学上等价计算的技巧, 在第二步计算中, 在矩阵乘中直接计算转置: 此时 t 轴变为 -1 轴, 而归约轴变为 -2/-3 轴。
- 实现细节可参考: https://gitcode.com/cann/cann-recipes-infer/blob/master/ops/pypto_python/impl/hc_pre_pypto.py

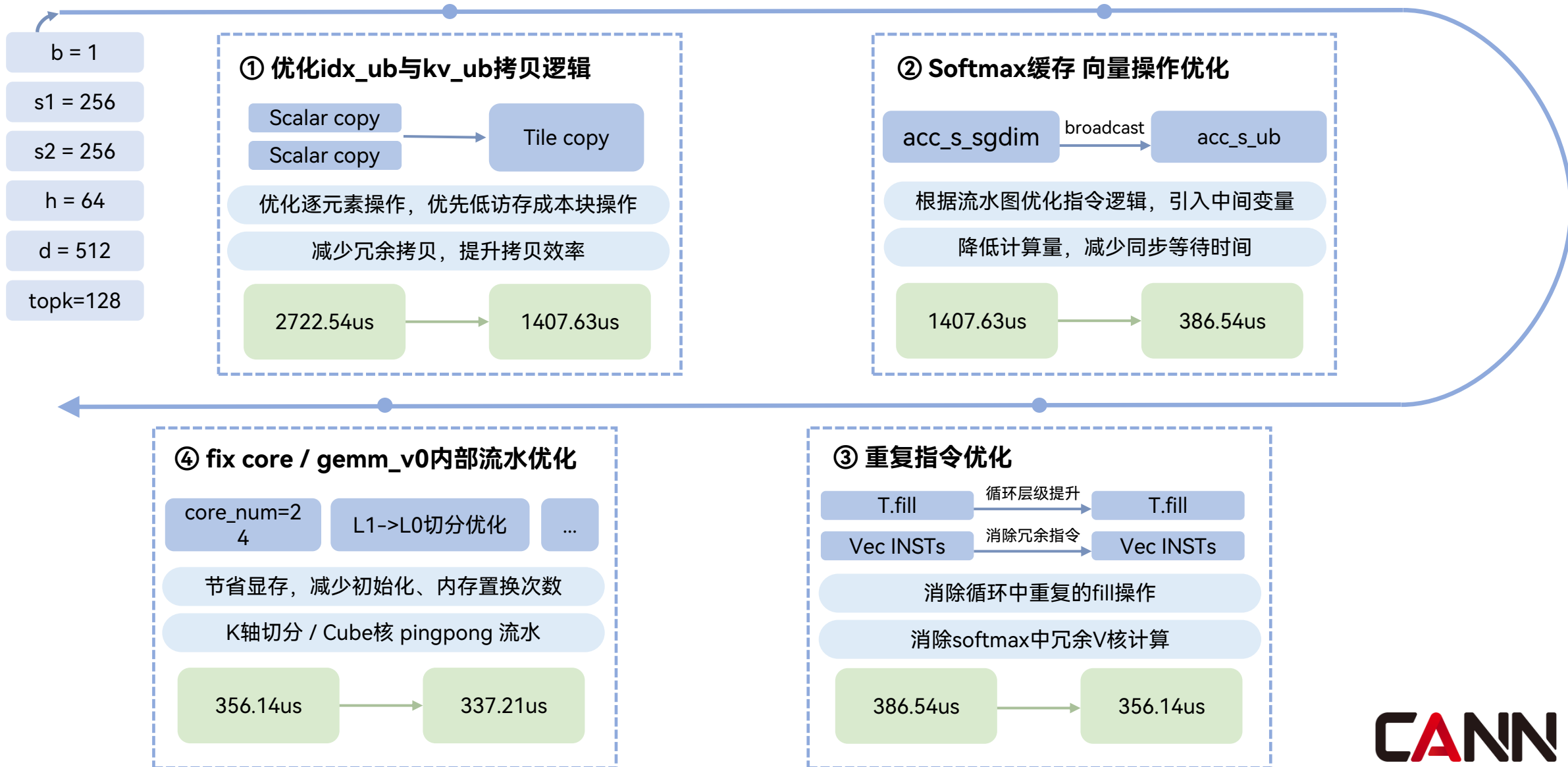
目录

Part1 整体介绍

Part2 主要算子实现

Part3 算子性能与调优

TileLang 性能优化案例：Sparse Attention 优化

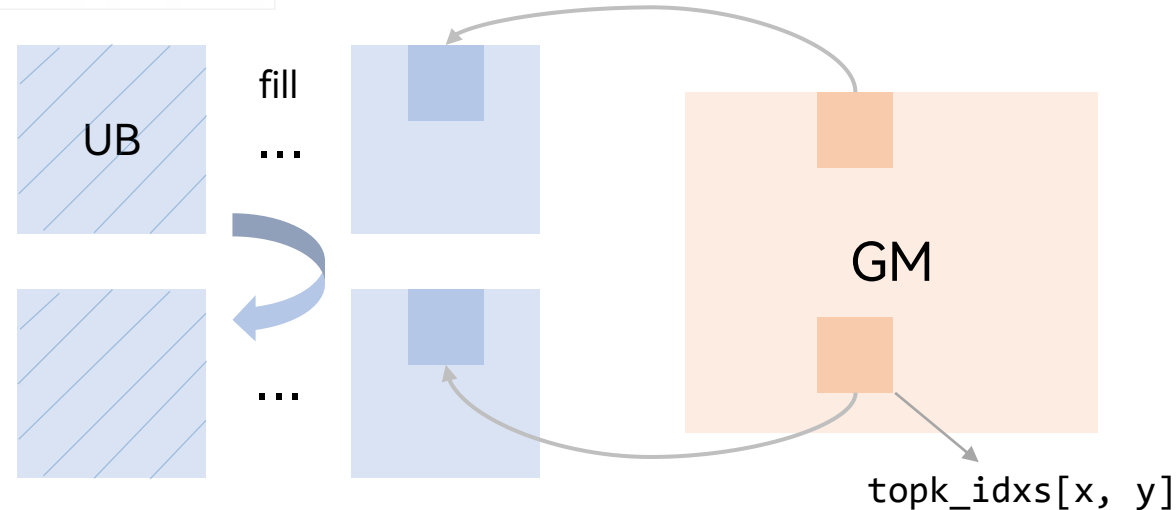


TileLang 性能优化案例: Sparse Attention 优化

$$idxs[i] = \begin{cases} topk_idxs[by, bx, t \cdot block + i] & \text{if } t \cdot block + i < topk \\ -1 & \text{otherwise} \end{cases} \quad \forall i \in [0, block)$$

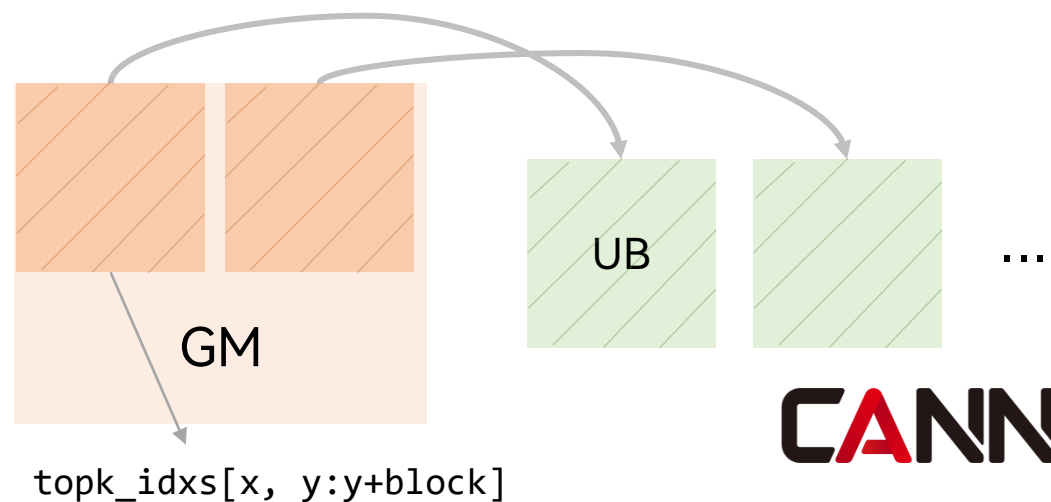
优化前: 低效 GM 逐元素读取

```
for t in T.serial(num_blocks):
    T.tile.fill(idxs_ub, -1)
    for i in T.serial(block):
        if t * block + i < topk:
            idxs_ub[i] = topk_idxes[by, bx, t * block + i]
```



优化后: 高效逐块搬运, 替代逐元素操作

```
for t in T.serial(num_topk):
    T.copy(topk_idxes[by, bx, t*block:t*block+block], idxs_ub)
    for i in T.serial(block):
        if t * block + i >= topk:
            idxs_ub[i] = -1
```



CANN

TileLang 性能优化案例：Sparse Attention 优化

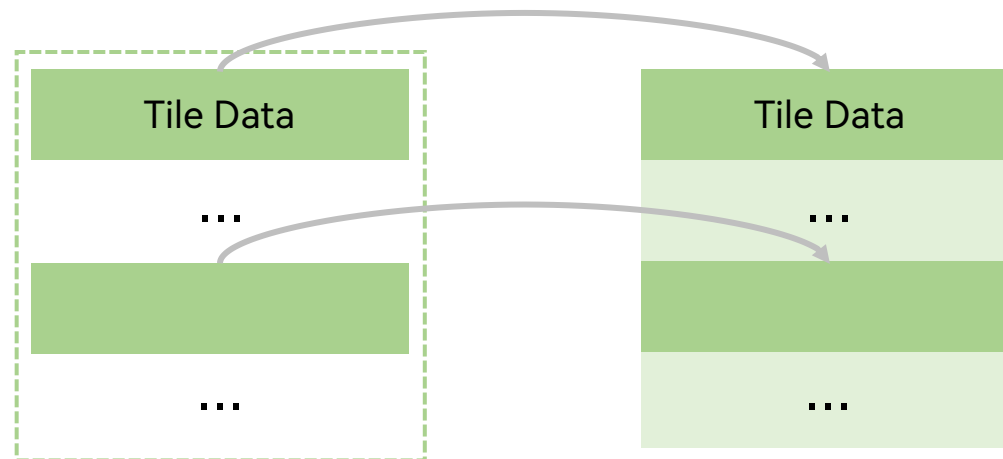
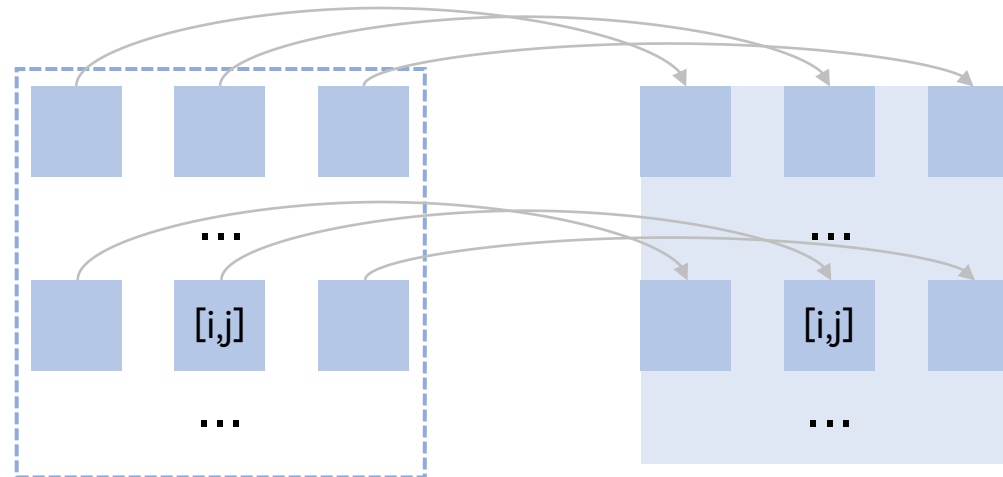
$$acc_s[i, j] = \begin{cases} 0 & \text{if } idxs[j] \neq -1 \\ -\infty & \text{otherwise} \end{cases} \quad \forall i \in [0, h), j \in [0, block)$$

优化前：逐元素赋值操作

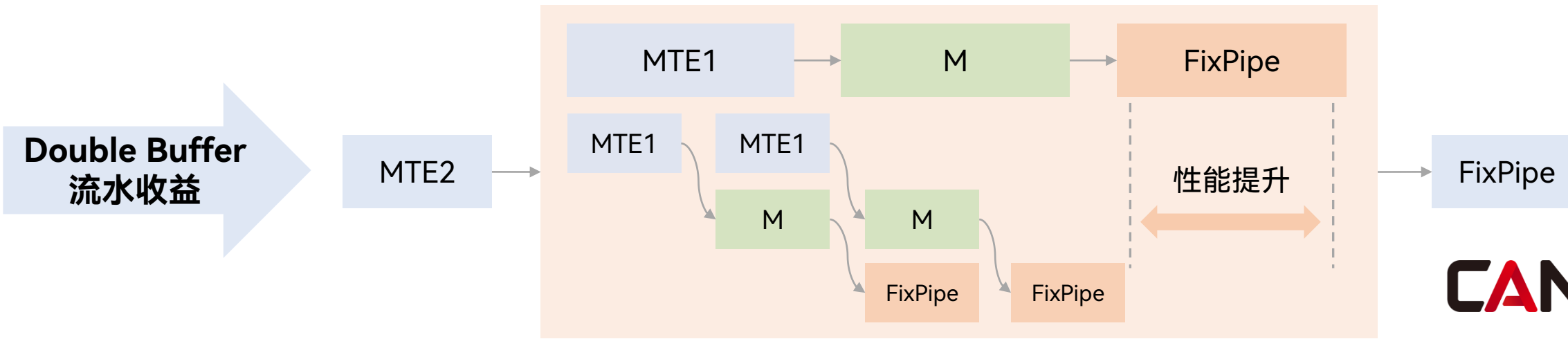
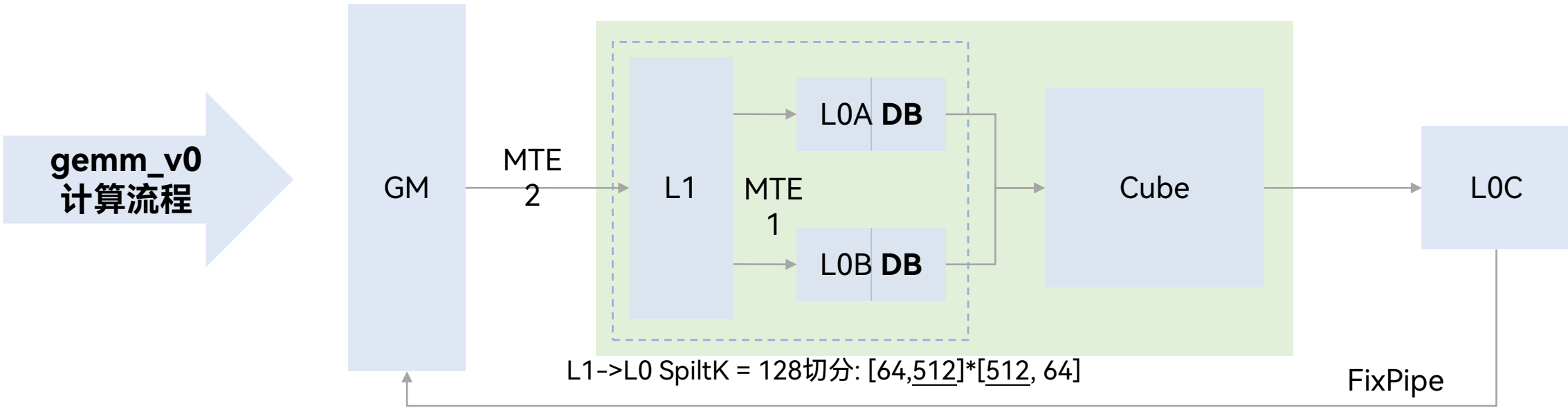
```
for i in T.serial(v_block):
    for j in T.serial(block):
        if idxs_ub[j] != -1:
            acc_s_ub[i, j] = 0
        else:
            acc_s_ub[i, j] = -T.infinity(accum_dtype)
```

优化后：DMA并行搬运，broadcast逐块操作

```
for i in T.serial(block):
    if idxs_ub[i] == -1:
        acc_s_ub singledim[0, i] = -T.infinity(accum_dtype)
    T.tile.broadcast(acc_s_ub, acc_s_ub_singledim)
```



TileLang 性能优化案例: Sparse Attention 优化



PyPTO: DeepSeek V4 算子性能

算子	KLOC	B	S1	S2	D	cmp_ratio	coff	start pos	Duration(us)
swa	0.3	64	2	8192	512	N/A	N/A	N/A	65
hca	0.2	64	2	8192	512	128	N/A	N/A	90
csa	0.2	64	2	8192	512	4	N/A	N/A	170
compressor_c4li	0.9	64	2	N/A	128	4	2	8192	55
compressor_c4a		64	2	N/A	512	4	2	8192	80
compressor_c128a		64	2	N/A	512	128	1	8192	55
compressor_c4li		64	2	N/A	128	4	2	8191	75
compressor_c4a		64	2	N/A	512	4	2	8191	95
compressor_c128a		64	2	N/A	512	128	1	8191	125
m1a_prolog	0.5	64	2	N/A	N/A	N/A	N/A	N/A	130
hc_pre	0.4	64	2	N/A	N/A	N/A	N/A	N/A	70
lightning_indexer_prolog	0.3	64	2	N/A	N/A	N/A	N/A	N/A	125

PyPTO 算子性能优化案例: Mla Prolog 性能优化

(1) 流水并行

PyPTO 的 MPMD 调度方式使开发者无需手动管理核间同步, 根据 Tile 切块依赖关系下发任务, 充分利用并行计算的机会

(2) 动态轴循环展开

Prefill/Decode 场景动态轴 t 变化范围大, 通过循环展开为静态图的方式, 实现多场景通用的较优性能

```
unroll_list = [128, 64, 32, 16, 1]
for tidx, unrollLength in pypto.loop_unroll(0, t, 1,
name="MLA_BS_LOOP",
idx_name="bs_offset", unroll_list=unroll_list, ):
```

(4) Vector 性能优化

通过设置合理的 TileShape, 使前后的 Vector 运算自动融合, 减少了冗余的 GM 和 UB 之间的数据搬运

```
pypto.set_vec_tile_shapes(4, 4096)
qr = rms_norm(q, attrs.eps)
qr = pypto.mul(qr, gamma_cq_2d_fp32)
qr_quant, qr_scale = quant(qr)
```

(3) Cube 性能优化

通过调整 TileShape, 实现大包搬运, 提升带宽利用率;
通过使能 k 轴切分, 提升核利用率;

```
pypto.set_cube_tile_shapes([tile_bs, tile_bs], [256, 512],
[128, 128], True)
kv = pypto.matmul(x_tile, wkv, pypto.DataType.DT_FP32)
```

PyPTO 算子性能优化: Mla Prolog 优化结果

优化细节1:

Query 路径: RmsNorm 和
DynamicQuant 自动融合

优化细节2:

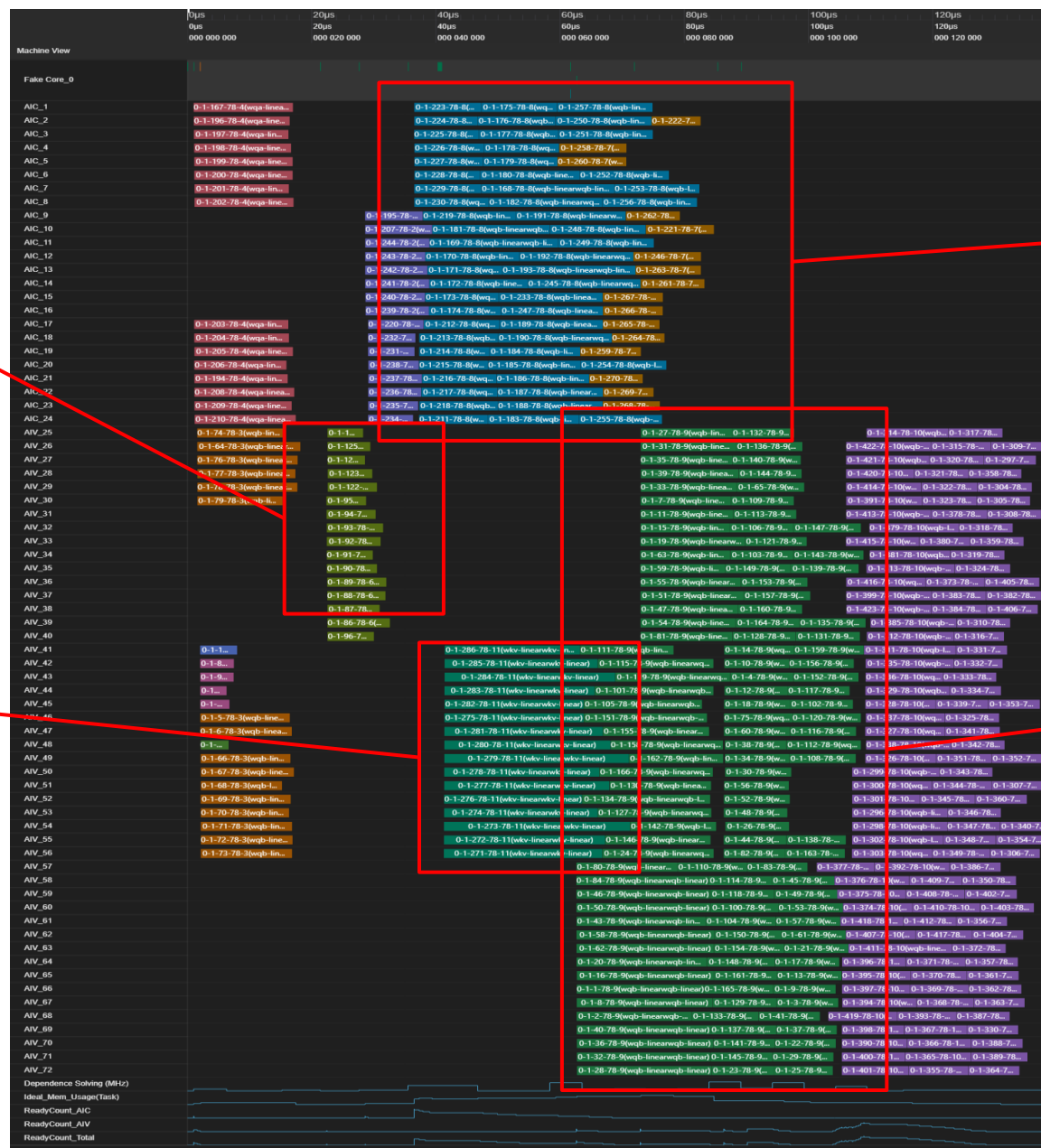
Key 路径: RmsNorm 和 RoPE
自动融合
和 Query 的矩阵乘并行执行

优化细节3:

(128, 1024) @ (1024, 32768)
int8 矩阵乘, 调整 TileShape 实
现高性能

优化细节4:

Dequant 和 RmsNorm 自动融
合, MPMD 调度使 Vector 计
算提前下发



PyPTO 算子性能优化案例：Compressor 性能优化

(1) 多场景性能优化

`Compressor` 模块使用的参数，例如压缩比 `compress\_ratio`、`rotate` 等配置，在各层之间有所不同。针对不同的场景配置，PyPTO 在同一个算子中根据不同的分支，在每个分支上进行各自的优化，从而在不同的使用场景下均能达到较为良好的性能。

(2) 计算图合并优化

不同 batch 的 vector 计算可能进入不同的分支且更新的 kv_state 和 score_state 的地址也不同。

通过设置“pg_skip_partition=True”，让一个batch的 vector 计算都在一个子图内。

```
for c_idx in pypto.loop(b_valid,
name="LOOP_COMP_2", idx_name="c_idx"):
    pypto.set_pass_options(pg_skip_partition=True)
```

(4) Vector 性能优化

通过合适的接口，避免冗余的 GM 和 UB 之间的数据拷贝；

合理的 TileShape 保证 Vector 运算自动合并

```
kv_state = scatter_update_3d(kv_state, index,
kv_pre)
score_state = scatter_update_3d(score_state,
index, score_pre)
```

(3) Cube 性能优化

动态 batch 轴，使用 b=64 进行切分，提升计算效率

K轴使能大包搬运，提升带宽利用率

```
b = 64
b_loop = (bsz + b - 1) // b
for b_idx in pypto.loop(b_loop, name="LOOP_COMP_1", idx_name="b_idx"):
    b_valid = (bsz - b_idx * b).min(b)
    x_view = pypto.view(x_tmp, [b * s1, h], [b_idx * b * s1, 0])
    pypto.set_cube_tile_shapes([128, 128], [256, 512], [128, 128])
    kv_t = pypto.matmul(x_view, wkv, pypto.DT_FP32, b_trans=True)
```

PyPTO 算子性能优化: Compressor 优化结果

优化细节1:

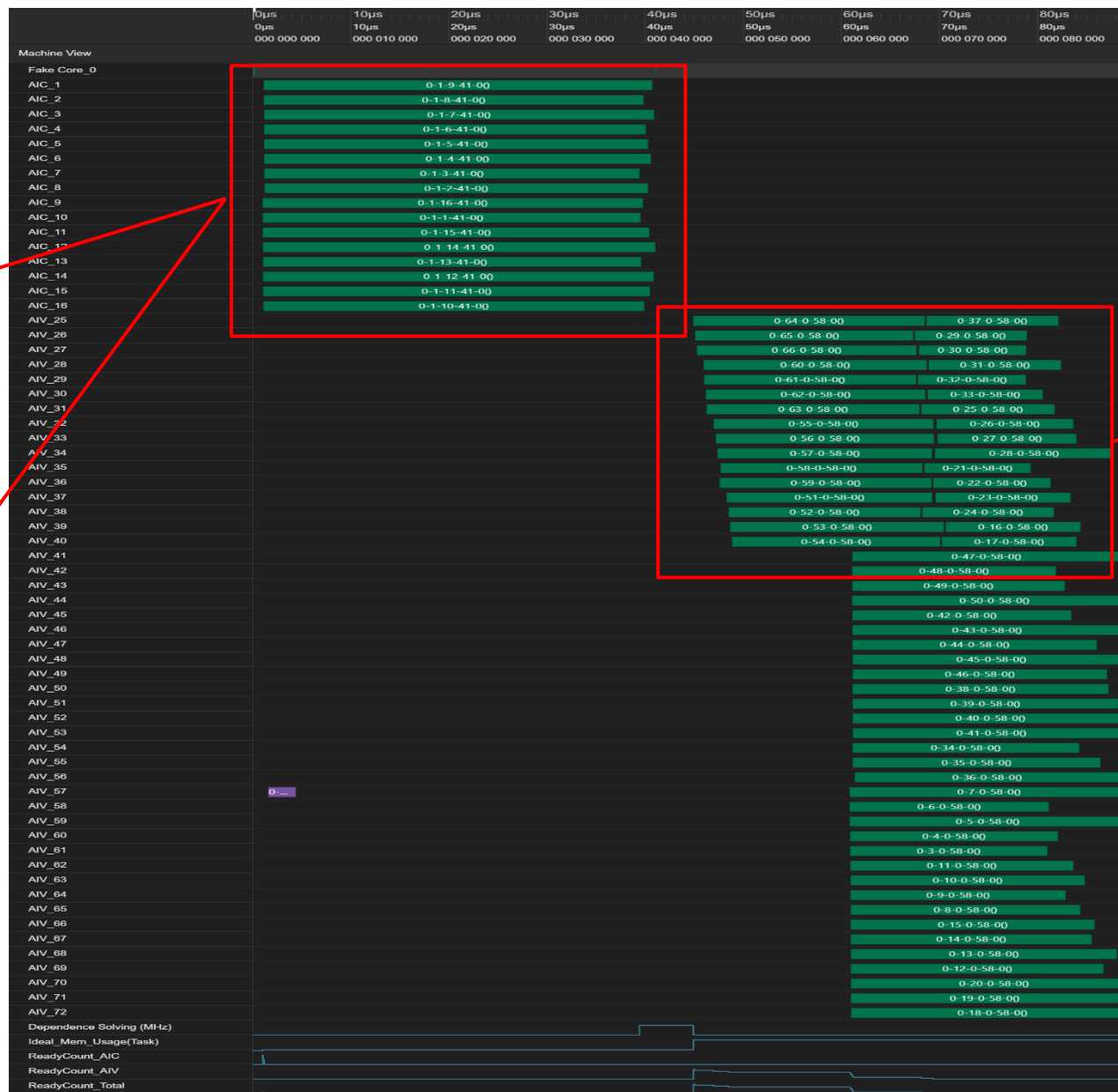
64 batch, MTP1, 且每个 batch 都触发压缩的场景

优化细节2:

两个(128, 4096) @ (4096, 1024) fp32 的矩阵乘, 通过 TileShape 调整和大包搬运实现良好性能

优化细节3:

64 个 batch, 切分成 64 个任务, 融合后避免冗余数据搬运



欢迎体验和贡献



上CANN社区获取干货



TileLang-Ascend社区



PyPTO 交流群

TileLang-Ascend 开源社区地址:

<https://github.com/tile-ai/tilelang-ascend>

PyPTO 开源社区地址:

<https://gitcode.com/cann/pypto>

CANN

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：以开源驱动技术创新，降低开发门槛，聚力构筑高性能
昇腾 AI 算力底座

Vision: Building an Open, Easy-to-Use, and Technology-leading AI
Computing Ecosystem

Mission: Drive technological innovation through open source, lower
development barriers, and jointly build a high-performance Ascend AI
computing infrastructure.



上CANN社区获取干货



关注CANN公众号获取资讯

<https://gitcode.com/cann>

CANN